

Name: _____ PUID: _____

Instructions and Policy: You are allowed to study with others and use online resources for reference, however, the work you turn in must be your own. This means do not copy/paste from Stack Exchange (or from another student.) If you have worked closely with other students, provide their name(s) and a brief (at most one paragraph) description of the interaction; if we feel this oversteps the bounds, we will discuss it with you. Each student should write up their own solutions independently.

The requirements below are supposed to be followed in this and further homework assignments.

- For your theoretical submission:
 - **YOU MUST INCLUDE YOUR NAME IN THE HOMEWORK.**
 - The answers **MUST** be submitted via Gradescope.
 - Please write clearly and concisely - clarity and brevity will be rewarded. Refer to known facts as necessary.
 - Theoretical questions **MUST include the intermediate steps to the final answer.**
- For your programming answers:
 - **The Python scripts will be submitted separately via Gradescope**
 - Zero points will be given in any question where the Python code answer doesn't match the answer on Gradescope.
 - If the answer is/includes a plot, it should be added to your theoretical submission unless otherwise specified.

Your code is **REQUIRED** to run on Python 3.11 in an environment detailed in Homework 0 under the Python Installation section. While your code may run in other environments, any points lost due to environmental issues will not be regraded.

Please make sure you don't use any libraries that are not imported for you. If such a library is used, you will get 0 pt for the coding part of the assignment.

If your code doesn't run on Gradescope, then even if it compiles on another computer, it will still be considered not running and the respective part of the assignment will receive 0 pt.

Total: 59 theory points + 45 programming points = 104 points.

Theoretical Questions (20+14+25=59 pts)

Please submit your answers on Gradescope.

Q1 (20 pts): True or False Questions

Answer the following as True or False with a justification or example. Points are uniformly distributed within the questions.

1. Consider two random variables with the following joint probability density function:

$$f(X, Y) = X + Y; \quad 0 \leq X \leq 1, \quad 0 \leq Y \leq 1.$$

Then, X and Y are independent.

2. If two events are independent, then they are also conditionally independent given any third event; and if two events are conditionally independent given a third event, then they are also (unconditionally) independent. (If False, give a counterexample for each direction.)

3. X is a random variable with $E[X] = 20$ and $E[X^2] = 425$. Then $V(3X - 2) = 225$.

4. If $P(A \mid B, C) = P(A \mid C)$, then A and B are conditionally independent given C , that is, $P(A, B \mid C) = P(A \mid C) P(B \mid C)$.

5. In the K-Nearest Neighbors algorithm, most of the computational cost occurs at training time, because the distances between all pairs of training instances must be precomputed before any prediction can be made.

Q2 (14 pts): Probability Review

1. (4 pts) Independence

Three fair six-sided dice D_1, D_2, D_3 are rolled independently. Define the random variables

$$X = \mathbf{1}[D_1 + D_2 \text{ is even}], \quad Y = \mathbf{1}[D_2 + D_3 \text{ is even}], \quad Z = \mathbf{1}[D_1 + D_3 \text{ is even}],$$

where $\mathbf{1}[\cdot]$ equals 1 if the condition holds and 0 otherwise. Show that random variables X, Y, Z are pairwise independent but not mutually independent.

Hint: the sum of two integers is even if and only if they have the same parity (both even or both odd).

2. (6 pts) Bayes Theorem

A laptop manufacturer sources batteries from 3 suppliers. The probability that a supplier-1 battery fails within 2 years is 0.5, with the corresponding probabilities for supplier-2 and supplier-3 batteries being 0.3 and 0.1, respectively. Suppose that 25 percent of the batteries used come from supplier 1, 35 percent from supplier 2, and 40 percent from supplier 3.

1. What is the probability that a randomly chosen battery will fail within 2 years?
2. Given that a battery failed within 2 years, what is the conditional probability that it came from supplier j , $j = 1, 2, 3$?

3. (4 pts) Probability Distribution

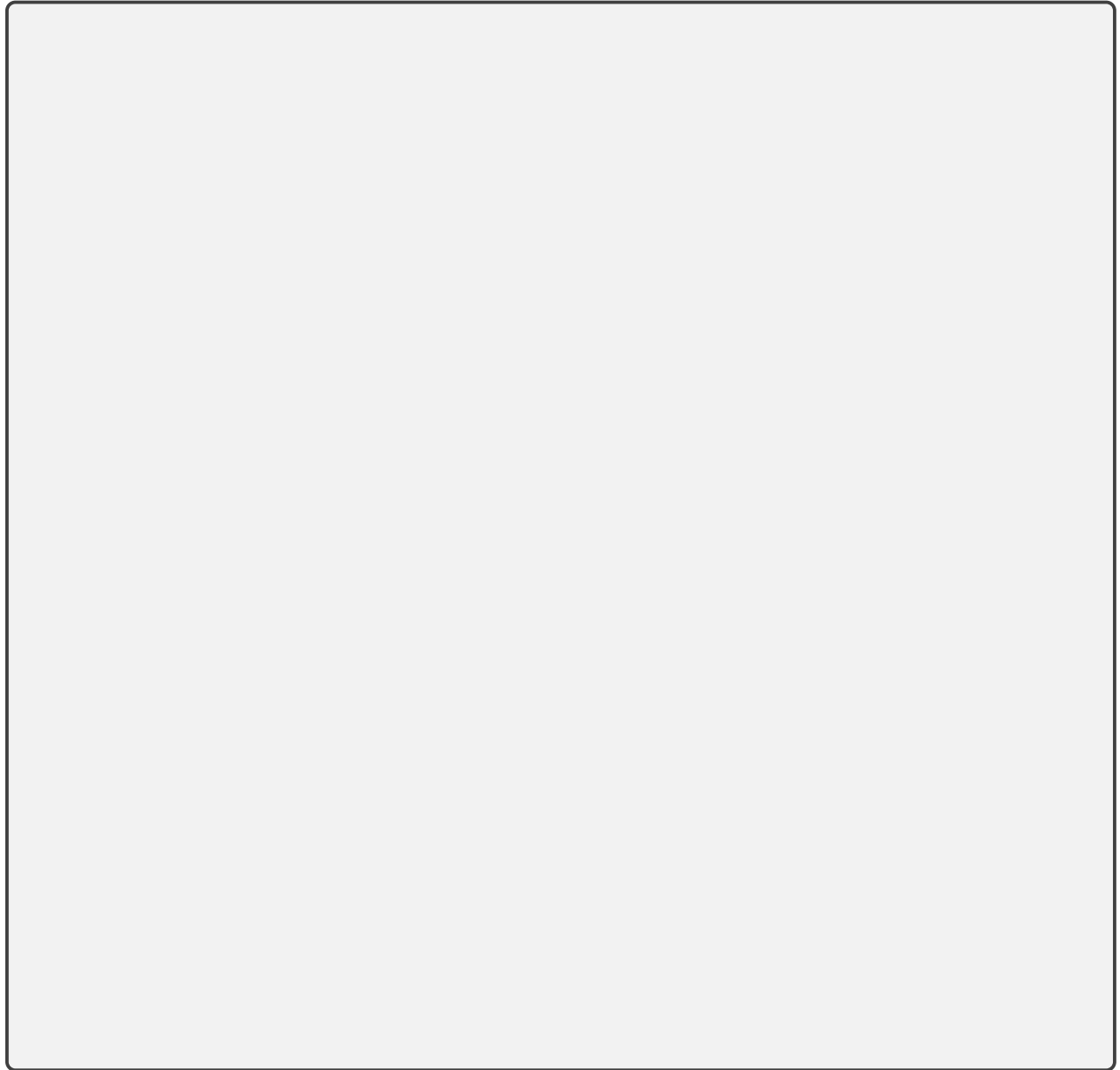
Consider two continuous random variables X and Y with joint probability density function

$$\begin{cases} f_{XY}(x, y) = \frac{1}{16}xy & 0 < x < k, \quad 0 < y < k, \\ f_{XY}(x, y) = 0 & \text{otherwise,} \end{cases}$$

Recall that a probability *density* function has the following properties:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{XY}(x, y) dx dy = 1$$

- (a) (1 pt) Compute the value of k so that $f_{XY}(x, y)$ is a valid joint probability density function.
- (b) (1 pt) Compute $P(Y > 2X)$: first express the integral in terms of k , then substitute the value from (a).
- (c) (1 pt) Compute $P(X + Y > 2)$: first express the integral in terms of k , then substitute the value from (a) (you may assume $k \geq 2$).
- (d) (1 pt) Using the k value obtained from (a), prove that random variables X and Y are independent (in case they are) or are NOT independent (in case they aren't).



Q3 (K-Nearest Neighbors 25 pts):

Consider a binary classification problem with labels $y \in \{+1, -1\}$. You are given the following training set in $\mathbb{R}^2$:

Index	Point $x = (x_1, x_2)$	Label
1	(1,1)	+1
2	(3,1)	+1
3	(1,3)	+1
4	(3,3)	-1
5	(150,150)	-1

Let the test point be

$$x_{\text{test}} = (2, 2).$$

And consider the following unless otherwise stated:

- Distance is Euclidean.
- If N points are tied at the k -th nearest distance, *all* such N points are included as neighbors, even if $N > k$.
- Ties in the final vote are broken by predicting label -1 .

(a) (**4 pts**) Compute the predicted label of x_{test} using **1-NN**, **3-NN**, and **5-NN**. Explicitly list the neighbors used in each case.

- (b) (**4 pts**) Suppose we rescale the first feature by a factor of 10, defining the transformation

$$\tilde{x} = (10x_1, x_2).$$

Recompute the predicted label of x_{test} using **3-NN** in the transformed space.

- (c) (**5 pts**) Suppose the training point $(150, 150)$ is replaced by **50 identical copies** of that same point, all labeled -1 . No other points are changed.

What is the prediction of **3-NN**, **4-NN**, and **6-NN** at x_{test} ?

- (d) (**6 pts**) Before applying **1-NN**, you may preprocess the data by applying a linear transformation

$$T(x) = Ax$$

to *all* points (training points and the test point), where $A \in \mathbb{R}^{2 \times 2}$ is a rank 1 matrix. Please note that each point in the feature space is a vector in $\mathbb{R}^2$, i.e., $x = (x_1, x_2)$. After applying T , Euclidean distance is used and **1-NN** is run in the transformed space, using the same tie-handling rules as above.

Is it possible to choose such an A so that the 1-NN prediction at x_{test} is -1 ? If yes, give an explicit matrix A and justify your reasoning/steps for finding it, but if no, explain why.

(e) (**6 pts**) Same as part (d), except now require that A is *full rank* ($\text{rank}(A) = 2$).

Is it possible to choose such an A so that the 1-NN prediction at x_{test} is -1 ? If yes, give an explicit matrix A and justify your reasoning/steps for finding it, but if no, explain why.

Programming Part (12 + 7 + 26 = 45 pts)

General Instructions

For this assignment, please download the provided `hw1.zip` file on **Brightspace**.

Overview

In this assignment, you will learn and implement:

- A KNN classifier
- Basics of data preprocessing
- Logistic Regression on the data you have preprocessed

Files

You will fill in functions in `knn.py`, `utils.py`, and `lr.py`. After you finish, you will submit these as well as `utils_knn.py` to Gradescope. The four submitted files are `knn.py`, `utils.py`, `lr.py`, and `utils_knn.py`.

Besides these three files, you will also find `utils_knn.py` and the dataset `pima-indians-diabetes.data.csv` in the same folder. You do not need to modify these files. `utils_knn.py` contains some helper functions used by `knn.py`.

Packages

You will need the following packages for this assignment:

- `numpy`
- `pandas`
- `matplotlib` (only for plotting in `lr.py`)

In particular, **you may not use scikit-learn anywhere in your submission**, even though it is installed in the grading environment. The autograder checks for this.

If you want, you can install `tqdm` to see a progress bar when running the autograder. This is not required for this class and might not be heavily focused on in future assignments, but it is a useful package to know if you are doing any machine learning work in Python.

These packages should be installed if you have installed Anaconda. If you are not using Anaconda, you can install them using `pip` or `conda`. For example, you can run `pip install numpy` in the terminal to install `numpy`.

Note that for this assignment, you should **not** use any other packages. You may see some other packages in the skeleton code, but they are only used for testing and grading. If you use any other packages, you may lose points. If you are not sure whether you can use a package, please ask the course staff.

Evaluation

Unless otherwise specified, your code will be evaluated by an autograder on Gradescope using Python 3.11 and the package versions specified in the provided `environment.yml`. You should make sure your code runs without errors in this environment. Otherwise, you may lose points. You can submit your code to Gradescope as many times as you want. We will only grade the latest submission.

There will be two types of test cases in the autograder: public (local) and hidden. Public test cases are visible to you, and you can see the results after you submit your code. You can also run the public test cases locally by running `python <filename>.py` in the same folder as your code. Passing all public test cases **does not** guarantee you will get full credit for the assignment.

Hidden test cases, however, are not visible to you, and you will not be able to see the results until your grade is published. The programming part is worth 45 points: 38 points from public and hidden autograder tests, 2 points for the plotting implementation, and 5 points for the report tasks described in the Logistic Regression evaluation section.

Getting Help

If you have any questions about this assignment, please contact the course staff for help, preferably during office hours. You can also post your questions on the course forum. However, please do not post any code publicly. When asking questions, you should describe your problem in words and post the relevant code snippet. Please avoid showing a screenshot of your code to TAs and ask “why my code is not working”.

1 KNN (1 + 1 + 2 + 2 + 4 + 2 = 12 pts)

In this part, you will implement a KNN classifier. From our lecture, you should know that KNN is a non-parametric classifier, which means it does not make any assumption about the underlying distribution of the data. Instead, it uses the training data directly to make predictions. In this assignment, you will implement the KNN classifier using the Minkowski distance with order 4 as the distance metric, and you will also implement a distance-weighted variant of the majority vote.

You will write your code in `knn.py`.

1.1 (1 pts) KNN - Constructor

Under `class KNearestNeighbor`, fill in the constructor (`__init__`) so that it takes in the number of neighbors `k` and a boolean flag `weighted` (default `False`) and stores them as attributes `self.k` and `self.weighted`. You should also initialize variables `self.X_train` and `self.y_train` to be `None`. These two attributes will be set in the `fit` method. The `weighted` flag is used in Q1.6.

1.2 (1 pts) KNN - Fit

Under `class KNearestNeighbor`, fill in the method `fit` so that it takes in the training data `X_train` and `y_train` and stores them as attributes `self.X_train` and `self.y_train`. Note that `X_train` is a 2D array of shape `(n_samples, n_features)`, and `y_train` is a 1D array of shape `(n_samples,)`. For this programming task, labels are nonnegative integers, as required by `np.bincount`.

The two `assert` statements are there to help you debug. The first one checks whether the number of training samples is the same as the number of training labels. The second one checks whether the number of training samples is greater than or equal to the number of neighbors `k`. If either of them fails, the supplied assertion raises an `AssertionError` with an appropriate error message.

1.3 (2 pts) KNN - Minkowski Distance

Under class `KNearestNeighbor`, fill in the method `calc_distance` so that it takes in a single test sample `x`, an integer `p` for the order of the distance, and returns a 1D array of shape `(n_samples,)` containing the Minkowski distance between `x` and each training sample in `self.X_train`.

About Minkowski distance: The Minkowski distance is a versatile metric used in statistics and machine learning to compute the distance between two points in a vector space. Defined by a parameter p , it generalizes several distances: Euclidean ($p = 2$) and Manhattan ($p = 1$). The distance between two points x and y is calculated as:

$$D(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (1)$$

where n is the number of dimensions. **In this homework, we will have $p = 4$.** Both `calc_distance` and `predict` default to $p = 4$. Your implementation must work for any integer $p \geq 1$ that is passed in; do not hard-code the value of p . Convert inputs to floating point before subtracting and raising differences to powers to avoid integer overflow.

1.4 (2 pts) KNN - Get Neighbors

Under class `KNearestNeighbor`, fill in the method `get_top_k` so that it takes in a 1D array `distances` and returns a 1D array of shape `(self.k,)` containing the indices of the top `self.k` smallest elements in `distances`. You may find the function `np.argsort` useful.

1.5 (4 pts) KNN - Predict

Under class `KNearestNeighbor`, fill in the method `predict` so that it takes in a 2D array `X_predict` of shape `(n_samples, n_features)` and returns a 1D array of shape `(n_samples,)` containing the predicted labels for each sample in `X_predict`. You should use the `get_top_k` method you implemented in Q1.4 to get the indices of the top `self.k` smallest elements in `distances`. Then, you can use these indices to get the corresponding labels from `self.y_train` and return the majority label among them as the predicted label. You may find the function `np.bincount` and the function `np.argmax` useful. Choose the smallest label when vote totals tie. In this programming task, select exactly k neighbors using `np.argsort`; the extra-neighbor tie rule from Theory Q3 does not apply.

1.6 (2 pts) KNN - Distance-Weighted Voting

In plain KNN every one of the k neighbors gets one vote, so a neighbor that is very close to the test point counts the same as one that is barely within the top k . A common refinement is *distance-weighted* voting: neighbor j at distance d_j from the test point votes with weight

$$w_j = \frac{1}{d_j + \varepsilon}, \quad \varepsilon = 10^{-8},$$

and the predicted label is the one with the largest *total* weight. Extend your `predict` method so that, when `self.weighted` is `True`, it uses this weighted vote instead of the plain majority vote (when `self.weighted`

is `False`, the behavior from Q1.5 must be unchanged). The `weights` argument of `np.bincount` may be useful. Think about a small example where the two voting rules give different answers; the doctest in `knn.py` contains one. Choose the smallest label if weighted vote totals tie.

2 Data Preprocessing (2 + 3 + 2 = 7 pts)

1. Introduction to Data Preprocessing

This part provides a brief introduction to data preprocessing. **No code is required for this introduction; complete the implementation tasks below.**

Data preprocessing is a crucial step in machine learning that involves preparing raw data for further analysis. It includes tasks such as cleaning, transforming, and organizing data into a usable format.

In this exercise, you will implement key preprocessing functions: splitting datasets into training and testing sets, standardizing features, and calculating the accuracy of predictions.

1.2 Key Components of the Implementation

The following sections explain how each part of data preprocessing is implemented, including dataset splitting, standardization, and accuracy calculation.

Dataset Splitting Splitting the dataset involves dividing it into training and testing subsets. This separation allows for model evaluation on unseen data. In this assignment you will implement the split yourself with `numpy` (you may **not** call `sklearn.model_selection.train_test_split`), following exactly this procedure so that your split matches the autograder's:

1. Create a random generator `rng = np.random.default_rng(random_state)`.
2. Draw a random permutation of the row indices: `perm = rng.permutation(n_samples)`.
3. Let `n_test = int(np.ceil(test_size * n_samples))`.
4. The first `n_test` indices of `perm` form the test set, the remaining indices form the training set (keep both in the order given by `perm`).

Standardization Standardization transforms features to have zero mean and unit variance. The formula used is:

$$z = \frac{x - \mu}{\sigma}$$

Where z is the standardized value, x is the original value, μ is the mean, and σ is the standard deviation. Both μ and σ are computed on the *training* set only and then applied to both the training and the test set.

Accuracy Calculation Accuracy measures how well the predicted labels match the true labels. It is calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

2. Implementation of Data Preprocessing

You are given a file `utils.py`. Please read the comments and existing implementations for more details. You are supposed to complete the following methods:

1. (2 pts) Complete the `split_data()` function, which takes in feature matrix X and target vector y , splitting them into training and testing sets based on a specified test size, using the `numpy`-only procedure described above. The split must be reproducible: the same `random_state` must always give the same split.
2. (3 pts) Complete the `standardize()` function, which standardizes both training and test datasets using the mean and standard deviation of the training set. To avoid a division-by-zero error, any feature whose standard deviation is exactly 0 must use `1e-7` in place of the standard deviation (this is checked by the autograder).
3. (2 pts) Complete the `accuracy()` function, which calculates the accuracy score by comparing true labels with predicted labels.

3 Logistic Regression (1 + 2 + 2 + 2 + 2 + 1 + 3 + 3 + 3 + 2 + 1 + 4 = 26 pts)

1. Introduction to Logistic Regression

This part is a brief introduction about Logistic Regression. **No code is required for this introduction; complete the implementation tasks below.**

Logistic regression is a popular algorithm for binary classification problems, where the goal is to assign an instance to one of two possible classes. The model estimates the probability of a data point belonging to a class, using the *sigmoid function* to map predictions to probabilities in the range $[0, 1]$.

Given an input feature vector X and model parameters W (weights) and b (bias), logistic regression computes the weighted sum of inputs as:

$$z = X \cdot W + b$$

The output of the model is given by the sigmoid function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where $\hat{y}$ represents the probability that a data point belongs to class 1.

1.2 Key Components of the Implementation

The following sections explain how each part of logistic regression is implemented, including the calculation of the loss function, gradients, and parameter updates during training.

Sigmoid Function The sigmoid function maps the real-valued output z into the $[0, 1]$ interval:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function is applied element-wise to the linear combination of input features X , weights W , and bias b .

Logistic Loss Function The logistic loss (also known as binary cross-entropy loss) is used to measure how well the model's predicted probabilities match the true labels. The mean loss over the samples is:

$$L_{\text{base}} = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i + \epsilon) + (1 - y_i) \log(1 - \hat{y}_i + \epsilon)]$$

Where m is the number of samples; y_i is the true label for the i -th sample; $\hat{y}_i$ is the predicted probability for the i -th sample; ϵ is a small constant to avoid numerical instability (e.g., 1×10^{-15}).

L2 Regularization To prevent overfitting, we add an L2 penalty term to the logistic loss. The L2 regularization encourages smaller weights, reducing the model's complexity:

$$L_2 = \frac{\lambda}{2} \sum_{j=1}^n W_j^2$$

Where λ is the regularization parameter, and n is the number of features. Note that only the weights W are penalized; the bias b is **not** part of the penalty. The total loss is the sum of the base logistic loss and the L2 regularization term:

$$L = L_{\text{base}} + L_2$$

Gradient Calculation To optimize the model parameters, we compute the gradients of the loss function with respect to the weights W and bias b . These gradients are then used in the gradient descent algorithm to update the parameters.

Gradient w.r.t. Weights The gradient of the loss function with respect to the weights is:

$$\frac{\partial L}{\partial W} = \frac{1}{m} X^T (\hat{y} - y) + \lambda W$$

Where $\hat{y} = \sigma(X \cdot W + b)$, and λW is the gradient of the L2 regularization term L_2 defined above. m is the number of samples.

Gradient w.r.t. Bias The gradient of the loss function with respect to the bias is:

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

(the regularization term does not depend on b , so it contributes nothing here).

2.5 Gradient Descent Update Once the gradients are computed, the model parameters are updated using gradient descent. The weights and bias are updated as:

$$W = W - \alpha \cdot \frac{\partial L}{\partial W}$$

$$b = b - \alpha \cdot \frac{\partial L}{\partial b}$$

Where α is the learning rate, controlling the step size of the update.

1.3 Training Process

During training, the model uses full-batch gradient descent: each epoch computes the gradients using the entire training set and updates the weights and bias once. The objective is to minimize the logistic loss by continuously adjusting the parameters until the loss converges or a specified number of epochs is reached.

1.4 Epoch and Convergence

An *epoch* refers to one full pass through the training data. During each epoch, the model computes the predictions, loss, and gradients, and then updates the parameters. The training process may stop early if the loss converges, i.e., the change in loss between consecutive epochs is below a specified tolerance.

2. Implementation of LR

You are given a file `lr.py`. In this Python file, you are supposed to implement the `LogisticRegressionClassifier` which takes four hyperparameters: the max number of epochs allowed, the learning rate, and the tolerance, and the regularization parameter `l2_penalty`. Please read the comments and the existing implementations for more details. You are supposed to complete several methods:

1. (1 pts) Complete the `sigmoid()` method, which takes in an array input and calculates the output of a standard logistic function given the input.

2. (2 pts) Complete the `predict_proba()` function, which takes in the training samples as input and return the probabilities of each label for every training sample.
3. (2 pts) Complete the `predict()` function, which takes in the training samples and predict their labels by calling `predict_proba()`. Probabilities greater than or equal to 0.5 should be predicted as class 1, otherwise 0.
4. (2 pts) Complete the `base_logistic_loss()` function, which calculates the base loss from the true labels and predicted probabilities.
5. (2 pts) Complete the `l2_regularization_loss()` function, which calculates the L2 loss $L_2 = \frac{\lambda}{2} \sum_j W_j^2$ as defined above (including the factor of $\frac{1}{2}$; the bias is not included).
6. (1 pts) Complete the `logistic_loss()` function, which adds the base logistic loss and the L2 regularization loss.
7. (3 pts) Complete the `gradient()` method, which takes in the training samples and their labels to calculate the gradient of the (regularized) loss with respect to the weights and the bias, using the formulas above.
8. (3 pts) Complete the `train_one_epoch()` function, which takes in training samples, their labels, then applies one epoch of gradient descent by updating `self.weights` and `self.bias` with the values computed in question (7).
9. (3 pts) Complete the `train()` function, which takes in training samples and validation samples with their labels. You should call `self.train_one_epoch()` in each iteration. After that, compute the training loss, validation loss, training accuracy, and validation accuracy using functions that are already implemented. You may find the `self.logistic_loss()` method in `lr.py` and the `accuracy()` function in `utils.py` useful. **Note: if the absolute difference between the training losses of two consecutive epochs is smaller than the tolerance, you need to stop training.**
10. (2 pts) Complete the `plot_logistic_regression_curve()` method, which takes in the training and validation accuracies and losses as input and should plot the relationships between the training and validation accuracies and losses vs. training epochs. The function will finally save the plot as `learning_curve_lr.png` under your working directory.

3. Evaluation

After the implementation of the `lr.py`, you are supposed to directly run `lr.py` code to do two things (the script trains on the Pima Indians Diabetes dataset, `pima-indians-diabetes.data.csv`, which is included in the handout):

- (1 pt) Attach the plotted learning curve to a small report, which should be submitted on gradescope together with the theoretical question submissions (we will designate a spot for a question submission with respect to this report). The figure will be generated under your working directory if your implementation of `plot_logistic_regression_curve()` function is correct.
- (4 pt) Try at least 4 combinations of hyperparameters and visualize the results using `plot_logistic_regression_curve()` function. You may try as many as you prefer, but only describe 4 runs and configurations you find informative. **DO NOT try hyperparameters at random.** Look for reasonable hyperparameter settings that may enable the model to perform better. Explain your reasoning in your hyperparameter search, attach the LR result plots of each hyperparameter setting, and talk about your findings in the report. You should try your best to outperform the baseline run, but if you fail to do so, as long as your search and results are well justified in the discussion, you will get full points.